CAPSTONE PROJECT REPORT

InsightCommerce

An AI-Powered E-Commerce Analytics and Decision Intelligence Platform

Course Title: Deep Learning

Submitted by

MD. Shahriar Ahamed Ridoy
ID: 261-25-008
M.Sc. in CSE (Major in Data Science)
Daffodil International University

Aisha Afroj
ID: 261-25-007
M.Sc. in CSE (Major in Data Science)
Daffodil International University

Submitted to

Sadat Hasan
Adjunct Faculty
Department of Computer Science and Engineering (CSE)
Faculty of Science and Information Technology (FSIT)
Daffodil International University

Submission date: 17 August 2026

Group capstone evidence: a verified 108,300-row dataset, 21 passing tests, a 10/10 query benchmark, and filtered aggregation below 3 ms on our development Mac.

Abstract and project result

We built InsightCommerce to make a large e-commerce dataset easier to explore and explain. Our verified Kaggle file contains 108,300 synthetic electronics transactions from 2025. We check the real schema, create a data-quality profile, derive documented calendar and product-taxonomy fields, store optimized Parquet, and use DuckDB for fast aggregation. The schema-aware assistant turns plain-English questions into a strict JSON plan and read-only SQL. Before anything runs, an AST safety gate blocks data changes, external access, sensitive-shaped fields, unrestricted star selection, excessive rows, and long execution. The application also keeps five successful conversation turns and allows one corrective retry. Its Streamlit interface includes global filters, preset insights, eight chart types, AI chart suggestions with manual override, and PDF, DOCX, CSV, HTML, and PNG export paths. We added daily demand prediction and Isolation Forest anomaly detection as advanced features. Our evaluation produced 21 passing tests, 10/10 benchmark answers, a 2.52 ms median filtered aggregation, and a held-out R-squared of 0.995. We report these results together with the limitations of using synthetic, single-year data.

Measured completion summary

Evidence	Result	Acceptance
Source integrity	108,300 rows; 0 missing; 0 duplicate IDs	PASS
Filtered aggregation	Median 2.52 ms	PASS (<500 ms)
Automated verification	21 tests + static quality checks	PASS
Question benchmark	10 of 10 questions	100%
Dashboard	5 sections; 8 prepared chart types	PASS
Advanced features	Demand prediction + anomaly detection	PASS

Central design choice: adapt to the real ten-column file. Profit, discount, shipping, and cancellation are not present, so no synthetic analytical field was invented.

1. Introduction, objectives, and dataset scope

For this capstone, we wanted to go beyond a dashboard that only displays fixed charts. Our goal was to let a user move from a business question to a result they can inspect, visualize, and export. That required a reproducible data layer, controlled AI-generated queries, interactive charts, and advanced analytics that stay honest about the limits of the dataset.

Objectives

- Verify the downloaded file before implementation and preserve the original unchanged.

- Deliver fast filtered aggregation, schema inspection, and measurable data-quality evidence.

- Convert natural language to safe DuckDB SQL with bounded retry and conversational context.

- Provide at least six interactive visual forms plus AI selection and human override.

- Add two advanced features: demand prediction and anomaly detection.

- Package tests, deployment configuration, documentation, benchmarks, final report, and ZIP.

Dataset scope

Dimension	Verified value
Period	2025-01-01 to 2025-12-31
Markets	15 countries
Products	105 electronics products
Customers	10,830 synthetic customer identifiers
Order value	$9.99 to $16,191.00; exact price x quantity
Total simulated revenue	$133,323,271.88

Although names and email addresses are synthetic, they are treated as sensitive-shaped data. They remain in the preserved source but are excluded from prompts, general exports, anomaly results, and the generated-query allowlist.

Project team

Member	Student ID	Program
MD. Shahriar Ahamed Ridoy	261-25-008	M.Sc. in CSE (Major in Data Science)
Aisha Afroj	261-25-007	M.Sc. in CSE (Major in Data Science)

We worked together across data preparation, application development, testing, documentation, and presentation planning.

2. System architecture

We organized the project into clear layers so that each result can be traced. DuckDB receives either parameterized dashboard filters or SQL that has already passed the AST safety check. Raw customer names and email addresses are never sent to an LLM.

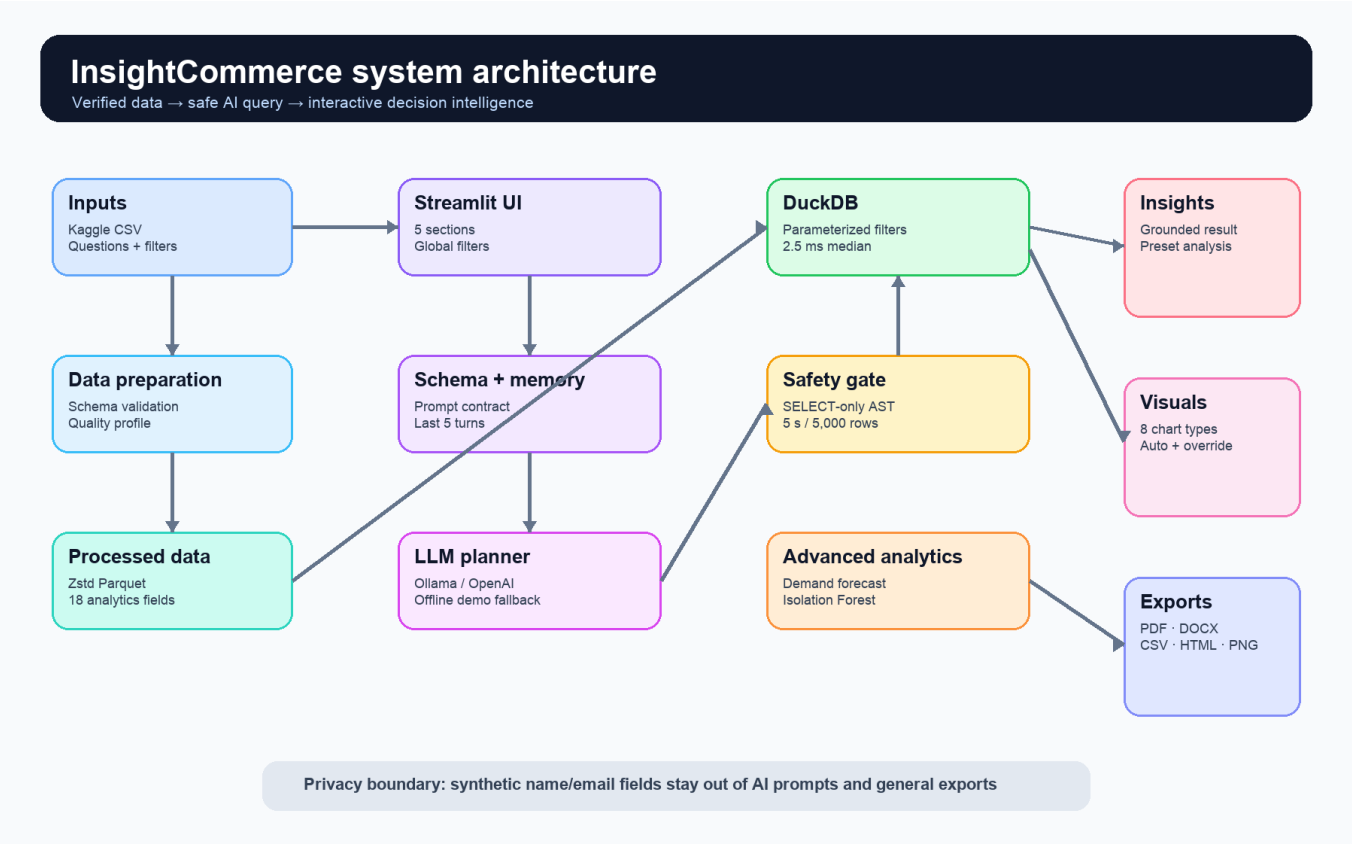

Figure 1. InsightCommerce component and trust-boundary architecture.

Layer	Responsibility	Primary evidence
Data	Validate, enrich, profile, convert, aggregate	quality_report.json; performance.json
AI query	Schema prompt, JSON plan, retry, memory	prompts.py; assistant.py
Safety	Parse, allowlist, cap, timeout, disable external access	sandbox.py; security tests
Experience	Filters, charts, explanations, exports	app.py; browser QA
Advanced	Demand forecast and anomaly ranking	ml.py; anomaly.py; model card

3. Task A - data backend and performance

We started by verifying the downloaded file instead of assuming its columns. The raw ZIP checksum is 81a80e884aeaf28f6816af25fa334dbcd712716337208194c5b451be339b63db. Our preparation command checks the exact column order, parses dates and numerics, strips identifier whitespace, enforces unique order IDs, confirms price x quantity, derives calendar/taxonomy fields, and writes Zstd-compressed Parquet plus JSON metadata.

Field	Type	Analytical use
order_id	string	Unique order count
date	date	Trend and seasonality
customer_id	string	Repeat-customer count
customer_name	string	Excluded from AI/general exports
customer_email	string	Excluded from AI/general exports
country	string	Geographic comparison
product	string	Product ranking and taxonomy
price	float	Unit-price distribution
quantity	integer	Units and transaction pattern
order_value	float	Simulated revenue

Performance result: 30 warm filtered aggregations produced a 2.52 ms median, 2.66 ms p95, and 2.71 ms maximum - comfortably below the 500 ms target.

4. Task B - schema-aware AI and safe text-to-code

When a user asks a question, we combine it with the public schema catalog and at most five previous successful turns. The model must return a strict QueryPlan containing SQL, chart type, result fields, a title, and a rationale. We never execute model-generated SQL directly.

Step	Control	Failure behavior
1	Question + schema + recent memory	No raw PII sample is sent
2	Strict JSON Schema query plan	Pydantic rejects missing/extra fields
3	sqlglot AST allowlist	Unsafe plans are blocked before DuckDB
4	5 s timeout + 5,000-row wrapper	Query is interrupted/capped
5	One error-informed corrective retry	Second failure becomes a clear user error
6	Grounded result narration	Only executed result values are described

Provider strategy

Mode	Purpose	Credential
Ollama	Local live LLM demonstration	None; local model required
OpenAI Responses	Optional hosted Structured Outputs	OPENAI_API_KEY
Offline demo	Deterministic common-question evaluation	None; clearly labeled fallback

Security tests prove that DROP, DELETE, COPY, external CSV reads, hidden schema access, multiple statements, sensitive fields, and unrestricted star selection are rejected.

5. Task C - interactive application

We divided the Streamlit application into Overview, Visual Explorer, AI Analyst, Prediction & Anomalies, and Reports & Quality. Date, country, and product-category filters apply consistently across the dashboard. The default overview remains legible at a tested 1600 x 1000 desktop viewport with no horizontal overflow or application exceptions.

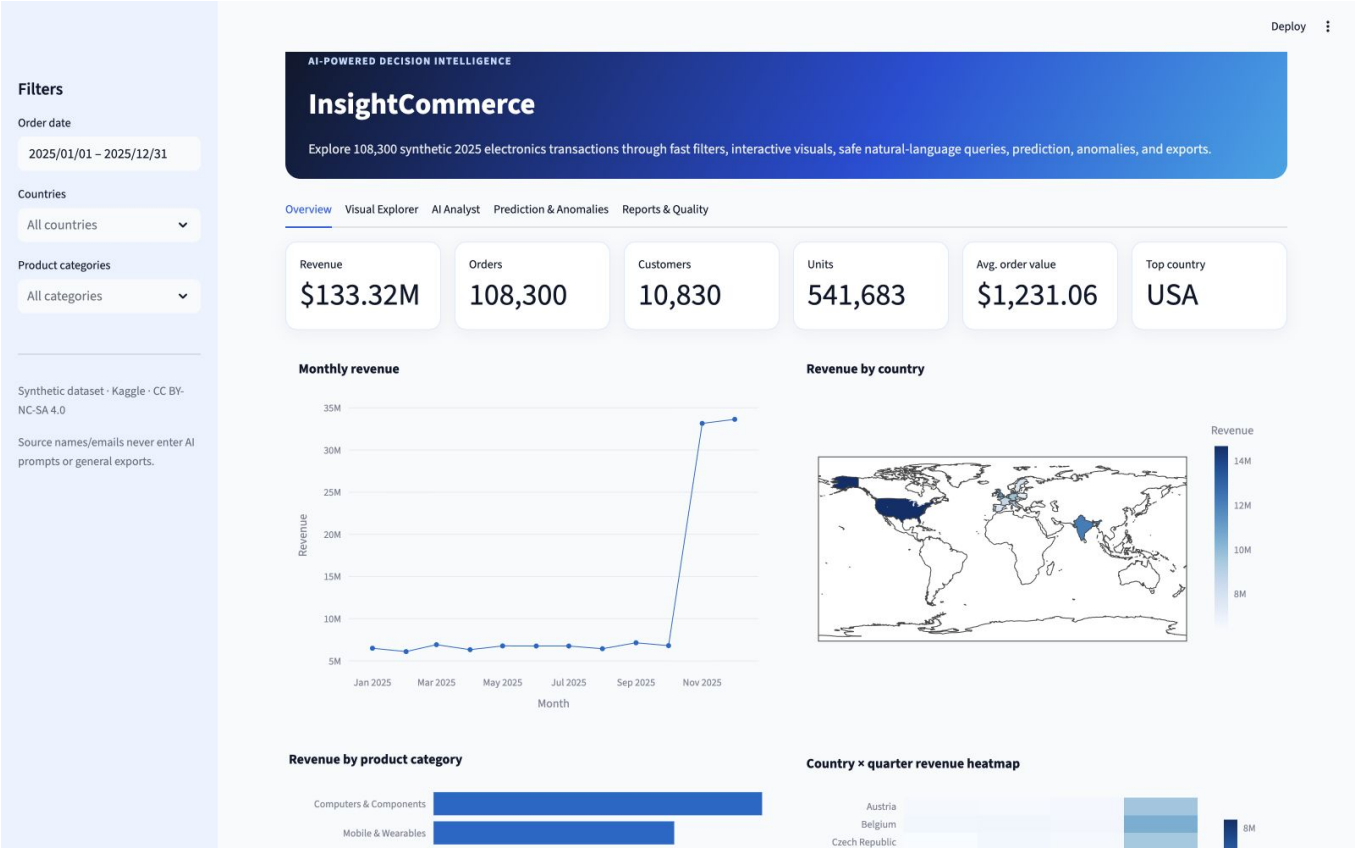

Figure 2. Browser-verified default dashboard on the development Mac.

Section	User outcome
Overview	KPIs, trend, map, category comparison, quarter heatmap
Visual Explorer	Eight prepared visual forms plus manual builder and chart downloads
AI Analyst	Question -> safe SQL -> table -> chart -> grounded explanation
Prediction & Anomalies	Forecast a date and investigate unusual orders
Reports & Quality	Inspect schema/quality and download PDF/DOCX summaries

6. Visualization and analytical evidence

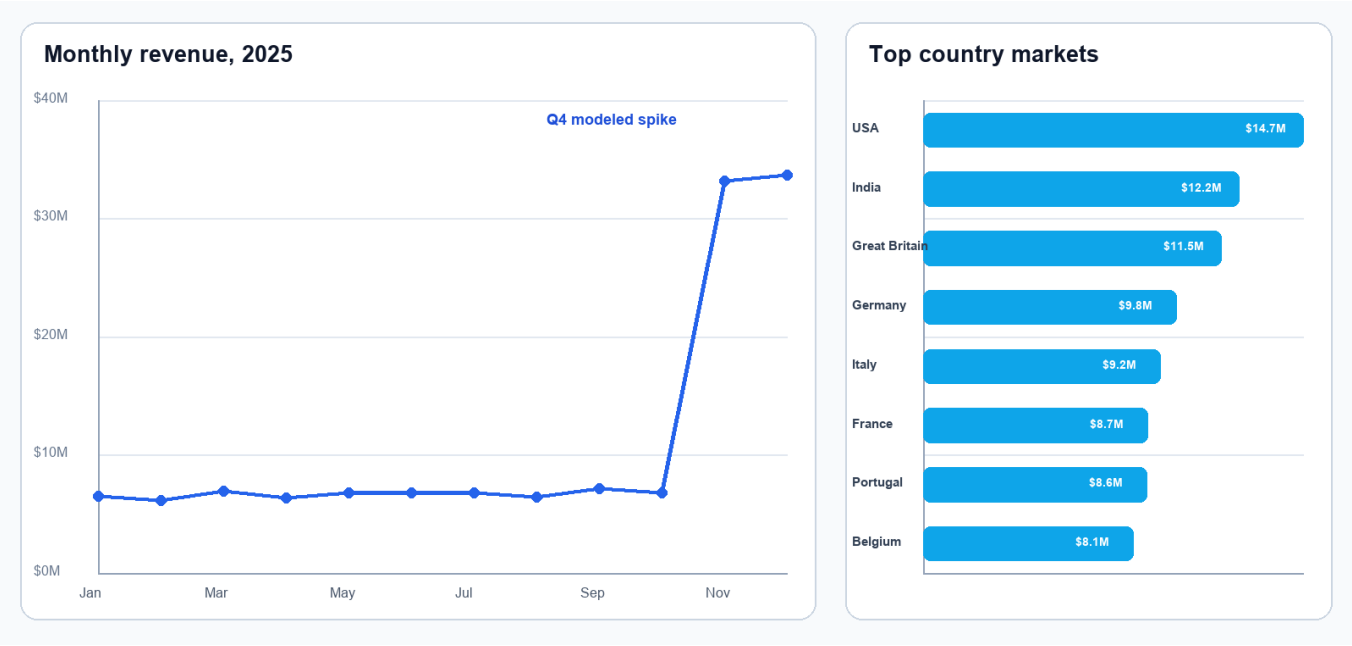

Figure 3. Measured 2025 revenue pattern and strongest country markets.

When we reviewed the yearly pattern, revenue stayed near $6-7 million per month from January through October, then rose above $33 million in both November and December. The source intentionally weights Q4, so this is a modeled pattern rather than evidence about a real company. USA leads total revenue because the synthetic generator also assigns different country record volumes.

Visual	Purpose	Interactivity
Line	Monthly revenue trend	Hover, zoom, range inspection
Choropleth	Country revenue	Country hover and scale
Bar	Category ranking	Sorted comparison
Heatmap	Country x quarter concentration	Color/intensity inspection
Treemap	Category -> product hierarchy	Drill and hover
Scatter	Price, quantity, order value	Multi-encoding exploration
Box	Price distribution by category	Outlier visibility
Histogram	Order-value distribution	Binned frequency

AI chart hints are accepted only when referenced columns and types are compatible. The user can override chart type and fields; incompatible combinations fall back safely.

7. Task D feature 1 - predictive analytics

Figure 4. Daily order-demand fit and model feature importance.

Metric	Held-out result	Interpretation
Test design	Every seventh day (53 days)	Seasonally distributed holdout
MAE	12.4 orders/day	Average absolute error
RMSE	18.2 orders/day	Penalizes larger misses
R-squared	0.995	Strong fit to synthetic seasonality
Model	Random Forest; seed 42	Reproducible nonlinear calendar model

Feature and prediction design

Known-in-advance features are month, quarter, weekday, day of month, annual sine/cosine, weekend, and Q4. The interface predicts daily order count for a chosen date. Indicative revenue is then predicted orders multiplied by the historical median order value; it is explicitly labeled as an assumption rather than a separate learned target.

The high R-squared reflects the dataset's designed Q4 step change. It must not be generalized to real retail demand without multi-year real data and rolling-origin tests.

8. Task D feature 2 - anomaly detection

Figure 5. Isolation Forest anomaly scores with flagged points in red.

Element	Implementation
Model	Isolation Forest, 160 trees, seed 42
Features	log price, quantity, log order value, value / product median
Nominal contamination	1%
Observed flags	1,006 of 108,300
Observed rate	0.93%
Output	score, flag, and transparent reason code

Interpretation boundary

Reason codes identify exceptionally high order value, premium unit price, maximum quantity, values well above a product's median, or a broader unusual multivariate combination. The output excludes name and email. A flag is an investigative signal - not proof of fraud, data error, or misconduct. Score ties can make the observed rate slightly different from the configured contamination.

9. Evaluation, testing, and browser QA

Verification area	Evidence	Result
Data contract	Exact schema, shape, identity checks	PASS
Generated SQL safety	7 malicious/unsafe cases + safe COUNT(*)	PASS
Retry behavior	Unsafe first plan, valid second; no third	PASS
Conversation memory	Seven writes retain only latest five	PASS
Charts	Time/country inference and invalid-hint fallback	PASS
Prediction/anomaly	Finite metrics, output, privacy columns	PASS
Exports	PDF, DOCX, CSV, interactive HTML signatures	PASS
Performance	Median 2.52 ms vs 500 ms	PASS
Static quality	Ruff across app, source, scripts, tests	PASS
Total automated tests	21	PASS

Browser walkthrough

We launched the application on our development Mac and tested it through the browser. We opened all five primary tabs and both nested advanced/report tabs. The offline monthly AI question executed in one attempt at 6.70 ms. An initial duplicate Plotly element ID was observed between hidden tabs, fixed with unique keys, and reverified. Final snapshots showed zero Streamlit exception elements and no horizontal overflow at 1600 x 1000.

Evaluation principle: pass/fail claims in this report come from committed JSON/Markdown artifacts, automated tests, or the final browser walkthrough.

10. Ten-question accuracy benchmark

We used ten representative questions to test the full offline path: natural-language plan selection, strict plan parsing, SQL safety validation, DuckDB execution, chart compatibility, and grounded narration. Each result is compared with independently executed reference SQL.

#	Question	Attempts	Time	Status
1	What is the total revenue?	1	0.65 ms	PASS
2	How many orders are there?	1	0.35 ms	PASS
3	How many units were sold?	1	0.48 ms	PASS
4	Show the monthly revenue trend for 2025.	1	2.42 ms	PASS
5	Which countries generated the most revenue?	1	2.49 ms	PASS
6	Which country has the highest average order value?	1	2.23 ms	PASS
7	Show the top 10 products by revenue.	1	2.90 ms	PASS
8	Compare revenue across product categories.	1	2.40 ms	PASS
9	Compare every quarter and highlight Q4 performance.	1	1.08 ms	PASS
10	What are the leading products by revenue?	1	2.73 ms	PASS

Per-question criterion	Required
Exact numeric/tabular result	Yes
Required result columns	Yes
Compatible chart recommendation	Yes
Success without corrective retry	Yes
Non-empty grounded narrative	Yes

Final benchmark accuracy: 10/10 (100%). Live LLM accuracy remains provider/model dependent and should be evaluated separately before production use.

11. Security, privacy, and reliability

Risk	Control	Residual limitation
Prompt injection	Question/history labeled untrusted; system contract fixed	A model may still propose bad SQL; validator is authoritative
SQL mutation	AST permits one SELECT/CTE only	Parser/library updates require regression tests
File/network access	Functions blocked; DuckDB external access disabled	OS/container permissions remain defense-in-depth
Sensitive-shaped fields	Prompt schema omits name/email; validator rejects them	Raw synthetic source still contains these columns
Resource exhaustion	5 s interrupt, 5,000 rows, 1 GB DuckDB limit	Host-level limits should also be configured
Hallucinated insight	Narrative is generated from executed result values	Live-model explanatory text still needs user judgment
Secret exposure	Environment/Streamlit secrets; real files ignored by Git	Operators must rotate compromised keys

Reliability controls

- Fixed random seeds make prediction and anomaly results reproducible.
- Parameterized application filters keep user values out of SQL strings.
- Only successful answers enter conversation memory.
- AI chart hints are type-checked before rendering.
- The original CSV and ZIP remain unchanged and attributable under their own license.
- CI reruns data preparation, tests, static checks, and both benchmarks.

12. Reproducibility and deployment preparation

We included pinned requirements and the raw data so a clean environment can reproduce the application. Parquet and quality files are rebuilt rather than treated as hidden inputs.

```
python3.12 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
python scripts/prepare_data.py
pytest
python scripts/benchmark_performance.py
python scripts/run_question_benchmark.py
streamlit run app.py
```

Prepared deployment targets

Target	Included support	AI behavior
Local macOS	Virtual environment + Streamlit	Ollama, OpenAI, or offline demo
Streamlit Cloud	app.py, runtime.txt, pinned requirements	Offline demo or hosted secret
Docker	Dockerfile + health check	Configure reachable protected provider
Process host	Procfile	Provider via environment variables
GitHub CI	Test, lint, data prep, two benchmarks	No external LLM required

Version control

Development is divided into regular milestone commits: verified data backend; safe schema-aware AI engine; dashboard and advanced analytics; submission documentation and evaluation; and final report/package. This avoids a prohibited single-commit submission and makes review traceable.

13. Limitations, future work, conclusion, and references

Limitations

- The source is synthetic and represents only 2025; findings are demonstrations, not company facts.

- Order value is deterministically price x quantity; no profit, cost, discount, shipping, or returns exist.

- Country volumes and Q4 behavior were modeled by the dataset creator and drive several results.

- The forecast holdout is seasonally distributed, not a multi-year rolling-origin production test.

- Isolation Forest identifies unusual combinations, not fraud or operational error.

- Live LLM results depend on provider availability, model behavior, latency, and cost.

Future work

If we continue this work with instructor approval and a richer real dataset, we would add product cost/profit, discount, shipping, return status, and customer segment fields; evaluate rolling multi-year forecasts; add authenticated user roles and audit logs; benchmark multiple live LLMs; and deploy managed observability with rate limits and query traces.

Conclusion

By completing InsightCommerce, we met Tasks A-D in one integrated and testable application. For us, the most important result is not one chart or model; it is the controlled path from a verified file to generated code, checked execution, and an exportable answer. The project is fast, visually complete, privacy-conscious, reproducible, and clear about what this synthetic dataset can and cannot support.

References

1. W. Kielbowicz, Synthetic E-Commerce Electronic Sales 2025 Dataset, Kaggle, 2026. https://www.kaggle.com/datasets/wojciechkiebowicz/e-commerce-electronic-sales-2025-dataset

2. DuckDB Documentation. https://duckdb.org/docs/

3. Streamlit Documentation. https://docs.streamlit.io/

4. Plotly Python Documentation. https://plotly.com/python/

5. scikit-learn User Guide: Random Forests and Isolation Forest. https://scikit-learn.org/stable/user_guide.html

6. OpenAI API Documentation: Structured model outputs. https://developers.openai.com/api/docs/guides/structured-outputs

7. Ollama Documentation. https://docs.ollama.com/

8. sqlglot documentation and source. https://github.com/tobymao/sqlglot